학습 노트: llm_judge 노드 — LLM severity 재판단 패턴

작성일: 2026-05-24 관련 파일: `dev2/nodes/llm_judge.py` (구현 예정), `dev2/prompts/severity_judge.py` (완성됨), `dev2/nodes/run_tools.py` (참고), `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 5 — LLM severity 재판단 LangGraph 노드 구현

---

0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저)

비유: 1차 채점 후 선생님의 재검토

CLI 도구(pylint, bandit, semgrep)는 빠른 자동 채점기입니다. 규칙에 맞으면 기계적으로 "이건 Critical, 저건 Minor" 라벨을 붙입니다. 그런데 자동 채점기는 문맥을 모릅니다. 예를 들어 `eval()` 사용을 무조건 Critical로 찍지만, 실제로는 테스트 코드 안의 안전한 사용일 수도 있죠.

`llm_judge` 노드는 경력 많은 선생님이 자동 채점 결과를 한 번 더 검토하는 역할입니다. 과거 우리 팀의 비슷한 사례(pgVector 검색 결과)를 참고해서, "이건 도구가 과하게 잡았네, Minor로 내리자" 또는 "이건 도구가 놓쳤네, Major로 올리자" 라고 등급을 재조정합니다.

파이프라인에서의 위치

```
run_tools_node          →  llm_judge_node          →  (Dev4로 전달)
(CLI 도구 원본 결과)        (LLM이 severity 재판단)
static_raw / security_raw  → enriched 필드 채움
```

- 입력: `state["static_raw"]`, `state["security_raw"]` (run_tools 노드가 채운 CLI 원본)
- 출력: `{"enriched": [...]}` — LLM이 재판단한 결과 (`Dev2State.enriched` 필드, schema.py:38)
- **LangGraph**가 이 **dict**를 **state**에 자동 **merge** 합니다 (run_tools_node와 동일한 반환 규칙)

> 핵심 분리: `severity_judge.py`는 프롬프트 문자열만 만듭니다 (LLM 호출 안 함). `llm_judge.py`는 그 프롬프트를 받아 실제 **LLM**을 호출하고 응답을 파싱합니다. 이렇게 나누면 프롬프트 로직은 LLM/API 없이 순수 Python으로 테스트할 수 있습니다.

개념 1. LangChain + Claude 통합

비유

`ChatAnthropic`는 **Claude**에게 전화를 거는 전화기입니다. 전화기 모델(`model=`)을 고르고, 할 말(`messages`)을 정한 뒤 통화 버튼(`invoke`)을 누르면, 상대방 대답(`.content`)이 돌아옵니다.

개념

`langchain_anthropic.ChatAnthropic`는 LangChain이 제공하는 Claude 전용 채팅 모델 래퍼입니다. LangChain의 표준 인터페이스(`invoke` / `ainvoke`)를 따르기 때문에, LangGraph 노드 안에서 다른 LangChain 컴포넌트와 똑같은 방식으로 쓸 수 있습니다.

왜 **LangChain** 래퍼를 쓰나**? (anthropic SDK** 직접 호출 대신**)**

- **LangSmith** 자동 추적: config로 callbacks를 주입하면 모든 호출이 LangSmith에 자동 기록됩니다 (CLAUDE.md §3 "예외 없음"). anthropic SDK를 직접 쓰면 이 추적을 수동으로 붙여야 합니다.
- 표준 메시지 타입: `HumanMessage`, `SystemMessage` 같은 공통 타입을 써서 다른 LangChain 코드와 호환됩니다.
- 모델 교체 용이: 나중에 `claude-sonnet-4-6` ↔ `claude-haiku-4-5` 같은 모델 라우팅(개념 3)을 깔끔하게 할 수 있습니다.

실제 코드

```
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

# 모델 인스턴스 생성 — model 이름은 팀 표준만 사용 (CLAUDE.md §9)
llm = ChatAnthropic(
    model="claude-sonnet-4-6",    # ⚠ OpenAI 금지. 팀 표준 Claude만 허용
    temperature=0,                # severity 판단은 일관성이 중요 → 0 (창의성 불필요)
    max_tokens=1024,
)

# 동기 호출
response = llm.invoke([HumanMessage(content=prompt)])
text = response.content           # ← 응답 텍스트는 .content 로 추출
```

⚠️ 팀 규칙 **(CLAUDE.md §9)**: LLM은 `claude-sonnet-4-6` 등 Claude 모델만. OpenAI 임포트(`from langchain_openai import ...`)는 PR 리뷰에서 반려됩니다. 임베딩/Vector DB도 마찬가지로 pgVector만 (Milvus 금지).

실무 팁 — **`temperature=0`**: severity 등급은 "정답이 하나"인 분류 작업입니다. 같은 입력에 매번 다른 등급이 나오면 안 되므로 `temperature=0`으로 무작위성을 제거합니다. 글쓰기/요약이라면 0.7 정도를 쓰지만, 판정 노드에서는 0이 정석입니다.

개념 2. USE_MOCK_LLM 패턴 (CLAUDE.md §10-3)

비유

실제 식당을 열기 전에 종이 인형으로 손님 응대 연습을 합니다. 종이 인형(Mock)은 진짜 손님과 똑같은 자리에 앉고 똑같은 주문서를 내밀지만, 음식값(API 비용)은 안 듭니다. 연습이 끝나면 스위치 하나(`USE_MOCK_LLM=false`)로 진짜 손님을 받습니다.

개념

`USE_MOCK_LLM` 환경변수로 실제 **LLM** 호출 / 가짜 응답을 코드 변경 없이 전환하는 패턴입니다. Mock 함수는 실제 LLM과 완전히 동일한 **JSON** 구조를 반환해야, 호출하는 쪽 코드가 진짜인지 가짜인지 몰라도 됩니다.

왜 중요한가**? (3**가지 이유**)**

1. 비용: 개발 중 노드를 수십 번 돌리는데 매번 실제 Claude를 호출하면 API 비용이 쌓입니다. Mock은 공짜입니다.
2. 속도: 실제 LLM은 응답에 1~3초 걸립니다. Mock은 즉시 반환 → 단위 테스트가 빠릅니다.
3. 테스트 독립성: CLAUDE.md §10-5 "단위 테스트는 실제 DB/LLM 없이 통과해야 함". CI 환경에 API 키가 없어도, 네트워크가 끊겨도 테스트가 돌아갑니다.

실제 코드

```
import os
import json

# 개발 중 기본값 true (CLAUDE.md §10-3: os.getenv("KEY", "default") 패턴)
USE_MOCK_LLM = os.getenv("USE_MOCK_LLM", "true").lower() == "true"


def _mock_llm_response(current_severity: str) -> str:
    """실제 LLM과 동일한 JSON 구조를 반환하는 가짜 응답.
    severity_judge.py 프롬프트가 요구하는 형식과 정확히 일치해야 함."""
    return json.dumps({
        "severity": current_severity,        # Mock은 원래 등급 그대로 통과
        "issue_type": {"Critical": "Vulnerability", "Major": "Bug", "Minor":
"Code Smell"}.get(current_severity, "Bug"),
        "suggestion": "[MOCK] 개발용 가짜 제안 — 실제 LLM 미호출",
        "resolution_example": "[MOCK] 실제 환경에서 교정 코드 예시가 제공됩니다.",
        "rationale": "[MOCK] USE_MOCK_LLM=true",
    }, ensure_ascii=False)


async def _call_llm(prompt: str, model: str, current_severity: str) -> str:
    if USE_MOCK_LLM:
        return _mock_llm_response(current_severity)   # 같은 자리에 종이 인형
    # 실제 호출 (개념 5에서 async 처리 설명)
```

```
llm = ChatAnthropic(model=model, temperature=0, max_tokens=1024)
response = await llm.ainvoke([HumanMessage(content=prompt)])
return response.content
```

실무 팁: Mock 응답을 만들 때 "성공 케이스"만 만들지 말고, 잘못된 **JSON**을 반환하는 **Mock**도 하나 두면 개념 4(재시도)를 테스트할 수 있습니다. 예: `return "이건 JSON이 아닌 일반 텍스트입니다"`.

개념 3. 모델 라우팅 전략 (비용 최적화)

비유

병원 분류(triage)와 같습니다. 응급실에 환자가 오면 중증도에 따라 다른 의사에게 보냅니다. 생명이 위급하면(Critical) 최고 전문의(sonnet), 치료가 필요하지만 급하진 않으면(Major) 일반 당직의(haiku), 가벼운 찰과상(Minor)이면 의사 없이 간호사 처치(LLM 호출 생략)로 끝냅니다.

개념

이슈의 **severity**별로 다른 모델을 쓰거나 아예 **LLM**을 건너뛰는 전략입니다. 모든 이슈를 가장 비싼 모델로 판단하는 건 낭비이고, 모두 싼 모델로 하면 중요한 판단의 정확도가 떨어집니다.

왜 **severity**별로 다른 모델을 쓰나?

severity	처리 방식	이유
Critical	`claude-sonnet-4-6`	보안 취약점/배포 차단급. 오판 비용이 가장 큼 → 가장 정확한 모델 필요
Major	`claude-haiku-4-5`	리팩토링 권고급. 빠르고 저렴한 모델로 충분
Minor	**LLM** 호출 생략	코드 스멜. CLI 도구 판단을 그대로 신뢰 → 비용/시간 0

Critical은 양이 적은데 중요도가 높고, Minor는 양이 많은데 중요도가 낮습니다. 비싼 모델을 "적게 중요한 곳에" 집중시키면 전체 비용을 크게 줄이면서 정확도는 유지됩니다.

실제 코드

```
# severity → 모델 매핑 (None이면 LLM 호출 생략)
SEVERITY_MODEL_ROUTE: dict[str, str | None] = {
    "Critical": "claude-sonnet-4-6",   # 가장 정확
    "Major":    "claude-haiku-4-5",    # 빠르고 저렴
    "Minor":    None,                  # LLM 생략, CLI 판단 그대로
}


async def _judge_one(issue: dict, skill_level: str, similar_cases: list[dict],
tech_stack: list[str] | None = None) -> dict:
    current = issue["severity"]
    model = SEVERITY_MODEL_ROUTE.get(current, "claude-haiku-4-5")  # 기본값 haiku

    if model is None:
        # Minor → LLM 건너뜀. CLI 결과를 그대로 enriched에 통과
        return {**issue, "rationale": "CLI 도구 판단 채택 (LLM 생략)"}

    # severity_judge.py의 프롬프트 빌더 재사용 (LLM 호출 안 하는 순수 함수)
    from dev2.prompts.severity_judge import build_severity_prompt
    prompt = build_severity_prompt(
        message=issue["message"],
        current_severity=current,
        developer_level=skill_level,
        similar_cases=similar_cases,
        tech_stack=tech_stack,
        file_name=issue.get("file", ""),
    )
    raw = await _call_llm(prompt, model=model, current_severity=current)
    return _parse_with_retry(raw, fallback=issue)   # 개념 4
```

실무 팁: 라우팅 표를 코드 상단 상수로 빼두면, 나중에 "Major도 sonnet으로 올리자" 같은 정책 변경을 한 줄로 처리할 수 있습니다. if/else 안에 모델명을 흩어 놓으면 변경 지점을 찾기 어렵습니다.

개념 4. json.loads() + 재시도 패턴 (CLAUDE.md §12)

비유

번역가에게 "결과를 표 형식으로 줘"라고 했는데, 가끔 줄글로 답하는 경우가 있습니다. 그럴 때 화내지 않고 "표 형식으로 다시 줘"라고 한 번 더 부탁합니다. 그래도 안 되면, 일단 원래 자료를 그대로 쓰고 넘어갑니다(작업 전체를 멈추진 않음).

개념

LLM은 "JSON으로만 답하라"고 해도 가끔 앞뒤에 설명을 붙이거나 형식을 어깁니다. 그래서 `json.loads()`가 `json.JSONDecodeError`를 던질 수 있습니다. CLAUDE.md §12 규칙: 캐치 → **1**회 재시도 → **2**회 실패 시 **fallback(**원래 **severity** 그대로**)** 반환.

왜 **LLM** 응답을 직접 **DB**에 저장하면 안 되나**?**

- LLM은 `"severity": "Crtical"`처럼 오타를 낼 수 있고, 정의에 없는 값(`"Blocker"`)을 만들 수도 있습니다.
- 검증 없이 저장하면 Dev4의 3-Way 판정(`Critical >= 1 → escalate`)이 깨지거나, DB 제약 조건 위반이 납니다.
- CLAUDE.md §6 보안 규칙: "LLM 응답을 직접 DB에 저장 시 반드시 파싱/검증 후 저장". 즉 파싱**(json.loads)** + 검증**(severity**가 허용값인지**)** 후에만 신뢰합니다.

실제 코드

```
import json
import logging

logger = logging.getLogger(__name__)
VALID_SEVERITIES = {"Critical", "Major", "Minor"}


def _parse_with_retry(raw: str, fallback: dict, retry_fn=None) -> dict:
    """LLM 응답 JSON 파싱 + 검증. 실패 시 원래 issue로 fallback."""
    for attempt in range(2):   # 최초 1회 + 재시도 1회 = 총 2회 (CLAUDE.md §12)
        try:
            parsed = json.loads(raw)
            sev = parsed.get("severity")
            if sev not in VALID_SEVERITIES:       # 검증: 허용값인지 확인
                raise ValueError(f"허용되지 않은 severity: {sev}")
            # 검증 통과 → 신뢰 가능한 결과
            return {
                **fallback,
                "severity": sev,
                "suggestion": parsed.get("suggestion", fallback.get("suggestion",
""")),
            }
        except (json.JSONDecodeError, ValueError) as e:
```

```
            # CLAUDE.md §12: 원본 응답을 로그에 남김 (나중에 디버깅용)
            logger.warning("llm_judge JSON 파싱 실패 (시도 %d): %s | 원본: %.200s",
                           attempt + 1, e, raw)
            if attempt == 0 and retry_fn is not None:
                raw = retry_fn()   # 1회 재시도 (LLM 다시 호출)
                continue
            break
    # 2회 실패 → fallback: CLI 도구 원래 severity 그대로 사용 (예외 안 던짐)
    logger.error("llm_judge 2회 파싱 실패 → CLI severity fallback 사용")
    return {**fallback, "rationale": "LLM 파싱 실패 — CLI 판단 유지"}
```

⚠️ 실무 함정: 재시도를 무한 루프로 돌리면 비용 폭탄 + 파이프라인 멈춤이 됩니다. CLAUDE.md §12는 명확히 "재시도 **1**회"로 한정합니다. 그리고 최종 실패 시에도 예외를 던져 전체를 멈추는 게 아니라, 원래 **severity**로 안전하게 진행합니다 (§10-4 "에러는 삼키지 않되, 로그 후 진행").

실무 팁 — **JSON** 추출 보강: 실전에서는 LLM이 ```` ```json ... ``` ```` 코드블록으로 감싸는 경우가 흔합니다. `json.loads` 전에 정규식으로 `{...}` 부분만 뽑아내는 전처리를 두면 1차 실패율이 크게 줄어듭니다.

개념 **5. asyncio + LangChain** 비동기 패턴

비유

이미 `run_tools.py`에서 "종업원 한 명이 여러 테이블을 동시에" 굴리는 패턴을 봤습니다. LLM 호출도 똑같습니다. Critical 이슈 3개를 LLM에 보낼 때, 하나씩 줄 세워(3초 x 3 = 9초) 기다리지 말고 동시에 보내서**(**약 **3**초**)** 한꺼번에 받아옵니다.

개념

- **`llm.ainvoke([...])`** (async invoke): LangChain이 제공하는 네이티브 비동기 호출. `await`로 기다리는 동안 이벤트 루프가 다른 이슈 호출을 진행합니다. → 이게 **1**순위 정석.
- **`llm.invoke([...])`** (동기): 호출이 끝날 때까지 이벤트 루프를 막습니다. async 노드 안에서 그냥 쓰면 다른 호출이 멈춰 병렬성이 사라집니다.
- **LangGraph** 노드는 **`async def`**여야 run_tools_node처럼 다른 async 노드/도구와 매끄럽게 연결됩니다.

왜 **`asyncio.to_thread()`**가 필요할 때가 있나**?**

대부분의 LangChain 모델은 `ainvoke`를 지원하므로 그걸 쓰면 됩니다. 하지만 만약 **`ainvoke`**를 지원하지 않는 동기 전용 함수(예: 일부 임베딩 라이브러리, 레거시 SDK)를 async 노드

에서 호출해야 한다면, run_tools.py에서 subprocess를 처리한 것과 똑같은 방법으로 감쌉니다:

```
# run_tools.py에서 이미 쓰는 패턴 (line 159) — 동기 함수를 스레드풀로 오프로드
result = await asyncio.to_thread(lambda: some_sync_llm_call(prompt))
```

asyncio.to_thread()는 동기 함수를 별도 스레드에서 돌리고 그 동안 이벤트 루프는 멈추지 않게 해줍니다. 즉 "동기 함수를 비동기인 척 감싸는" 도구입니다.

실제 코드 (노드 전체 골격)

```
import asyncio
from dev2.schema import Dev2State


async def llm_judge_node(state: Dev2State) -> dict:
    """LangGraph 노드: CLI 도구 결과의 severity를 LLM으로 재판단한다.

    흐름:
        1. static_raw + security_raw 합치기 (run_tools 노드 출력)
        2. 이슈별 severity 라우팅 (Critical→sonnet, Major→haiku, Minor→생략)
        3. asyncio.gather로 LLM 호출 병렬 실행
        4. JSON 파싱 + 검증 + fallback → enriched 반환
    """
    issues = state.get("static_raw", []) + state.get("security_raw", [])
    if not issues:
        return {"enriched": []}   # CLAUDE.md §12: 빈 입력도 예외 없이 진행

    skill_level = state.get("skill_level", "junior")   # §12: 미존재 시 junior 기본
값


    # similar_cases는 vector_search 노드가 채운다고 가정 (pgVector 검색 결과)
    similar = state.get("similar_cases", [])

    # ── LLM 호출 병렬 실행 (run_tools_node의 gather 패턴과 동일) ──
    tasks = [_judge_one(issue, skill_level, similar) for issue in issues]
    enriched = await asyncio.gather(*tasks, return_exceptions=False)

    return {"enriched": enriched}   # LangGraph가 state에 merge
```

실무 팁 — **ainvoke vs to_thread** 선택: - LangChain 모델 호출 → **ainvoke** (네이티브 비동기, 가장 효율적) - ainvoke가 없는 동기 라이브러리 → **asyncio.to_thread** (run_tools.py 방식) - 둘을 섞지 말 것. ainvoke가 있는데 to_thread로 감싸면 스레드를 불필요하게 낭비합니다.

실무 팁 — **LangSmith config 주입 (CLAUDE.md §10-6)**: callbacks는 노드 함수에 하드코딩하지 말고 graph.ainvoke(config=...)로 주입합니다. 그래야 통합 시 trace 프로젝트명을 환경별로 바꿀 수 있습니다.

3rd-Project-sub2 프로젝트에서

파일	역할
`dev2/nodes/llm_judge.py` (구현 대상)	LLM을 호출해 severity 재판단. 입력 `static_raw`/`security_raw` → 출력 `enriched`
`dev2/prompts/severity_judge.py` (완성)	프롬프트 문자열만 생성 (`build_severity_prompt`). LLM 호출 안 함 → 순수 테스트 가능
`dev2/nodes/run_tools.py` (참고)	async + `asyncio.to_thread` + `gather` 패턴의 살아있는 예시. llm_judge도 같은 골격
`dev2/schema.py`	`Dev2State.enriched` 출력 필드 정의, `Issue` Pydantic 검증 모델
`dev2/.env.example`	`USE_MOCK_LLM=true`, `ANTHROPIC_API_KEY` 설정 위치

통합 시 주의 **(Week 5)**

- `USE_MOCK_LLM=false`로 전환할 때, Mock과 실제 응답의 JSON 키가 정확히 같은지 다시 확인 (`severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale` — 5개 필드).
- LLM 호출이 추가되므로 LangSmith trace에 노드별 비용/지연이 찍힙니다 — 모델 라우팅 효과를 여기서 검증.

복습 퀴즈 (5문항)

1. (이해) `severity_judge.py`와 `llm_judge.py`의 책임을 한 문장씩으로 구분해 설명하세요. 왜 이렇게 나눴나요?
2. (적용) `USE_MOCK_LLM=true`일 때 Mock 함수가 반드시 지켜야 할 조건은 무엇인가요? 안 지키면 어떤 문제가 생기나요?
3. (적용) Minor 이슈에 대해 LLM을 호출하지 않는 이유를 비용/정확도 관점에서 설명하세요.
4. (비교) `llm.invoke()`와 `llm.ainvoke()`의 차이는? async 노드 안에서 `invoke()`를 쓰면 무슨 일이 생기나요?

5. (적용) LLM이 JSON 대신 일반 텍스트를 2번 연속 반환했습니다. CLAUDE.md §12에 따르면 이 노드는 어떻게 동작해야 하나요? (예외를 던져야 하나요?)

정답 힌트

1. severity_judge=프롬프트 생성(순수 함수, LLM 없이 테스트 가능) / llm_judge=실제 LLM 호출+파싱. 분리하면 프롬프트 로직을 API 없이 단위 테스트 가능.
2. 실제 LLM과 동일한 JSON 구조(키 이름까지, 5개 필드: `severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale`) 반환. 안 지키면 false로 전환 시 파싱 코드가 깨짐.
3. Minor는 양이 많고 중요도 낮음 → 호출 비용 대비 효용이 적음. CLI 판단을 그대로 신뢰.
4. invoke=동기(이벤트 루프 블록), ainvoke=비동기(다른 호출과 병렬 가능). async 노드에서 invoke 쓰면 병렬성이 사라져 호출이 직렬화됨.
5. 1회 재시도 후에도 실패 → 예외를 던지지 않고 원래 CLI severity로 fallback, 로그 기록 후 진행.